2025-2026 Spring System Programming Course Assigment Questions

Assignment Application and Submission Guidelines

The solutions to the two programming problems listed below must be carried out in accordance with technical reporting standards. The assignment can be completed in groups of a maximum of 2 people. Points to consider during the assignment preparation process are as follows:

1. Coding and File Structure

* Each problem should be analyzed and solved separately, independently of the others.
* The source code for each problem should be organized into a single source file for each problem. All files must be submitted in a single main directory (folder).
* Inline comments explaining the algorithm's operation and the functions' tasks should be added to the source code. Code readability and the presence of explanatory lines are important in the evaluation criteria.

2. Reporting Standards

* A cover page should be prepared for the assignment report.
* A single report file should be prepared for 2 assignments and placed in a single directory along with the source code files. It should be named using the group student numbers.
* The assignment report should include the solution logic for the relevant questions, the source code written, and detailed explanations of this code.
* To verify the performance and accuracy of the programs during operation, it is mandatory to include working outputs (screenshots) for each question in the report.

3. Evaluation Criteria

Assignments will be graded based on the full fulfillment of the technical requirements mentioned above, the structural integrity of the code, the academic language of the report, and the presentation format. Incomplete files, code without comment lines, or reports and code files without working program screenshots will not be evaluated.

Q1: Write a C program that performs operations on a dynamically allocated 2D matrix using pointer arithmetic only. You are given a matrix of size N×N . The matrix must be allocated dynamically using malloc.

Requirements

1. Dynamically allocate the matrix in the `main` function using a **single pointer (1D memory block)**.
2. Implement the following function:

```
int special_sum(int *mat, int rows, int cols);
```

3. Inside the function:

- Traverse the matrix **only using pointer arithmetic** (do NOT use `mat[i][j]` syntax)
- Compute the sum of:
  - The **main diagonal elements** (i==j)(i == j)(i==j)
  - The **secondary diagonal elements** (i+j==cols−1)(i + j == cols - 1)(i+j==cols−1)
- Ensure that the center element (if it exists) is **not counted twice**

4. In the `main` function:

- Read the matrix dimensions from the user
- Allocate memory dynamically using `malloc`
- Read matrix elements from the user
- Call the `special_sum` function
- Print the result
- Properly **free the allocated memory**

Important Notes:

- You must use **pointer arithmetic** to access matrix elements, Do not use 2D array indexing (`[][]`)

- Memory allocation must be done using `malloc`

Example Input/ Output:

Input:

```
1 2 3
4 5 6
7 8 9
```

Output: Sum : `25`

Q2: Write a C program that demonstrates signal handling using SIGALRM, SIGINT, SIGSTOP, and SIGCONTsignal call.

Requirements

1. The program must create a **child process** using `fork()`.
2. **Child Process:**
   - Runs an infinite loop and prints a counter every second
   - Handles:
     - `SIGINT` → prints *"SIGINT received but continuing..."*
     - `SIGCONT` → prints *"Process resumed"*
   - Note: `SIGSTOP` cannot be handled
3. **Parent Process:**
   - Uses `alarm(3)` to trigger every 3 seconds
   - On each alarm:
     - Sends `SIGSTOP` to pause the child

        - Waits 2 seconds
        - Sends `SIGCONT` to resume the child
    - After ~10 seconds:
        - Sends `SIGINT` to child
        - Terminates child and exits
4. Use only:
    - `signal()`
    - `alarm()`
    - `kill()`

SAMPLE OUTPUT

```
Child counter: 0
Child counter: 1
Child counter: 2
Parent: Stopping child...
Parent: Continuing child...
Child: Process resumed
Child counter: 3
Child counter: 4
Parent: Stopping child...
Parent: Continuing child...
Child: Process resumed
Parent: Sending SIGINT...
Child: SIGINT received but continuing...
Parent: Child terminated.
```